

Continuous State Space Models and S4

1. The Mathematical Formulation of Continuous State Space Models

When looking at a galaxy, the light hitting your eye is a continuous flow, not a set of files. This is the starting point of the whole theory. In the real world, things change smoothly (continuous) over time. To capture this, we use a **State Space Model** (SSM), which is just a fancy way of saying we use a hidden **memory** to turn an input into an output.

Imagine a **black box** where you feed in a signal. Inside the box, the model keeps a summary of what it has seen so far. This is described by two main equations:

$$\begin{aligned}x'(t) &= Ax(t) + Bu(t) \\ y(t) &= Cx(t) + Du(t)\end{aligned}$$

Variable Explanation:

- $u(t)$: **The Input.** This is the 1D signal coming in (like the light intensity from pixels).
- $x(t)$: **The Hidden State.** This is a vector of size N that acts as the model's memory. It stores a compressed version of all previous inputs.
- $x'(t)$: **The Derivative.** This tells us how the memory (x) is changing at this exact moment in time, based on the current state and new input.
- $y(t)$: **The Output.** This is the final signal produced by the model.
- A : **The State Matrix.** This is the **brain** of the system. It defines the internal dynamics: how the memory interacts with itself to keep or discard information.
- B : **The Input Matrix.** It determines how the new incoming data (u) is mapped into the memory (x).
- C : **The Output Matrix.** It decides which parts of the memory (x) are important to create the final result (y).
- D : **The Direct Term.** This is basically a skip-connection that lets the input go straight to the output without being stored in memory.

2. The Discretization Procedure (with Explicit Derivations)

Computers can't handle **continuous** calculus; they work in discrete steps (ticks of a clock). To make the SSM work on a processor, we have to turn those smooth equations into a step-by-step formula. This is called Discretization. The papers use the Bilinear Transform (also called Tustin's method) because it is very stable.

The Step-by-Step Derivation: We start by assuming a small time-step between samples, which we call Δ . We approximate the change in memory (x') as the difference between the next step (x_k) and the previous step (x_{k-1}) divided by the time-step:

$$\frac{x_k - x_{k-1}}{\Delta} = x'(t)$$

We also approximate the state $x(t)$ as the average of the two steps:

$$x(t) = \frac{x_k + x_{k-1}}{2}$$

Now, we plug these into our original state equation:

$$\frac{x_k - x_{k-1}}{\Delta} = A \left[\frac{x_k + x_{k-1}}{2} \right] + Bu_k$$

Multiply both sides by Δ to clear the fraction:

$$x_k - x_{k-1} = \left(\frac{\Delta A}{2} \right) x_k + \left(\frac{\Delta A}{2} \right) x_{k-1} + (\Delta B)u_k$$

Now, we want to get the **future** (x_k) on one side and the **past** (x_{k-1}) on the other. Move the x_k term from the right to the left:

$$x_k - \left(\frac{\Delta A}{2} \right) x_k = x_{k-1} + \left(\frac{\Delta A}{2} \right) x_{k-1} + (\Delta B)u_k$$

Factor out the x terms and use the Identity matrix (I), which acts like the number **1** for matrices:

$$\left(I - \frac{\Delta}{2}A \right) x_k = \left(I + \frac{\Delta}{2}A \right) x_{k-1} + (\Delta B)u_k$$

Finally, to isolate x_k , we multiply by the inverse of the left side:

$$x_k = \left[\left(I - \frac{\Delta}{2}A \right)^{-1} \left(I + \frac{\Delta}{2}A \right) \right] x_{k-1} + \left[\left(I - \frac{\Delta}{2}A \right)^{-1} \Delta B \right] u_k$$

Discrete Form Result:

$$\begin{aligned} x_k &= \bar{A}x_{k-1} + \bar{B}u_k \\ y_k &= Cx_k \end{aligned}$$

Variable Explanation:

- Δ : **The step size.** It tells the model how much **time** passes between samples.
- I : **The Identity Matrix.** A square matrix with 1s on the diagonal and 0s elsewhere.
- $\bar{A}$ and $\bar{B}$: These are the **discretized** versions of A and B . These are the actual constants you would use in a programming loop.

3. Motivation for Structured Parameterizations (HiPPO)

If you just pick random numbers for your A matrix, the model will have a very short memory. It will forget what happened lets say 100 steps ago. This is called the **vanishing gradient** problem.

To solve this, the S4 paper uses HiPPO (High-order Polynomial Projection Operators). HiPPO is a mathematical way to build the A matrix so that the state (x) is always trying to reconstruct the entire history of the signal. It does this by projecting the history onto a set of orthogonal polynomials. It matters because HiPPO provides a theoretical guarantee that the model can remember information from thousands of steps ago. Without this structure, the model wouldn't be able to handle long sequences like a 4,096-pixel galaxy image.

4. Diagonal Parameterization Strategy in S4D

The original S4 was quite complex because the A matrix was **Normal Plus Low Rank** (NPLR). While this worked, it was hard to implement and required a custom algorithm.

The S4D (Diagonal) paper simplifies this. It proves that you can actually get the same performance by making the A matrix Diagonal (every number in the grid is zero except for the middle diagonal line).

This matters because:

- **Speed:** Multiplying a vector by a diagonal matrix takes $O(N)$ time. Multiplying by a full dense matrix takes $O(N^2)$ time.
- **Memory:** You only need to store N numbers instead of $N \times N$.
- **Hardware Efficiency:** Diagonal matrices are much easier to handle on small processors (like RISC-V) because the math is simpler and requires fewer memory transfers. S4D shows that as long as you initialize the diagonal matrix using the HiPPO principles, it works just as well as the complicated version.

5. Computational Advantages of the Convolutional Formulation

S4 is unique because it can be used in two different ways depending on what you are doing.

The Recurrent View: For running the model (inference), we use the loop: $x_k = \bar{A}x_{k-1} + \bar{B}u_k$. This is great for hardware because the memory needed is **fixed**. No matter how long the image is, you only need to store the current state (x).

The Convolutional View: For training the model, loops are slow because you can't calculate step 10 until step 9 is finished. However, if you unroll the math of that loop, it turns into a single big **Convolution** operation. The whole output is just the input multiplied by a **Kernel** (K) (filter).

$$K = (C\bar{B}, C\bar{A}\bar{B}, C\bar{A}^2\bar{B}, \dots, C\bar{A}^{L-1}\bar{B})$$

The Advantages:

- **Parallel Training:** Because it's a convolution, we can use the Fast Fourier Transform (FFT). This lets the computer calculate the whole sequence at once in parallel, which is much faster than a loop.
- **Efficiency:** It allows the model to have the **infinite** context of a recurrent model but the **training speed** of a convolutional model.

Variable Explanation:

- K : **The SSM Convolution Kernel.** It is a long list of numbers that represents the model's entire memory filter.
- L : **The sequence length** (e.g., 4,096 pixels).
- FFT : **Fast Fourier Transform**, an algorithm that makes calculating convolutions extremely fast.